Київський національний університет будівництва і архітектури

Кафедра Кібербезпеки та комп’ютерної інженерії

МАГІСТРАТУРА

Лабораторна робота №2

Тема: Розробка ноди керування рухом робота через /cmd_vel

Виконав студент КСМм-26:
Павлюк О.А.

Перевірив
Маринський К.В.

Київ – 2026

Мета:

Сформувати практичні навички розробки ноди керування рухом у ROS2, навчитися працювати з повідомленням geometry_msgs/Twist, реалізувати підписку на /cmd_vel, обмеження швидкості та перевірити роботу ноди в симуляції.

Завдання:

1. Підготовка середовища

1.1. Активувати ROS2-середовище.

1.2. Перевірити працездатність команд:

- ros2 node list
- ros2 topic list

1.3. Запустити turtlesim або інший доступний симулятор для тестування.

2. Створення пакету Motion Control

2.1. Створити новий пакет типу ament_python для реалізації ноди керування рухом.

2.2. Додати залежність від geometry_msgs.

2.3. Зібрати workspace та перевірити коректність встановлення пакету.

3. Реалізація Motion Node

3.1. Створити ноду, яка підписується на Topic:
/cmd_vel

3.2. Використати тип повідомлення:
geometry_msgs/msg/Twist

3.3. Реалізувати обробку полів:

- linear.x
- angular.z

3.4. Виводити отримані значення через логер ROS2 (INFO-рівень).

4. Реалізація обмеження швидкості

4.1. Додати перевірку допустимих меж швидкості.

4.2. Якщо значення перевищує встановлений поріг — обмежити його.

4.3. У разі перевищення виводити WARN-повідомлення.

5. Тестування ноди

5.1. Запустити Motion Node.

5.2. Подати команди у /cmd_vel через teleop або через ros2 topic pub.

5.3. Перевірити, що нода отримує повідомлення та коректно їх обробляє.

5.4. Переконатися, що обмеження швидкості працює.

Звіт

1) Створюємо ноду **motion_node.py**

```python
class MotionControlNode(Node):
    def __init__(self):
        super().__init__('motion_control_node')

        #Підписка на топік /cmd_vel з типом geometry_msgs/msg/Twist
        self.subscription = self.create_subscription(
            Twist,
            '/cmd_vel',
            self.cmd_vel_callback,
            10
        )

        #Встановлення граничних значень швидкості
        self.max_linear_x = 0.5   # Максимальна лінійна швидкість (м/с)
        self.max_angular_z = 1.0  # Максимальна кутова швидкість (рад/с)

        self.get_logger().info('Motion Control Node успішно запущено та підписано на /cmd_vel')

    def cmd_vel_callback(self, msg: Twist):
        #Зберігаємо початкові значення
        linear_x = msg.linear.x
        angular_z = msg.angular.z

        #Перевірка та обмеження linear.x з WARN-повідомленням
        if abs(linear_x) > self.max_linear_x:
            self.get_logger().warning(
                f'Лінійна швидкість перевищує ліміт! Отримано: {linear_x:.2f}, обмежено до: {self.max_linear_x:.2f}'
            )
            linear_x = max(-self.max_linear_x, min(self.max_linear_x, linear_x))

        # Перевірка та обмеження angular.z з WARN-повідомленням
        if abs(angular_z) > self.max_angular_z:
            self.get_logger().warning(
                f'Кутова швидкість перевищує ліміт! Отримано: {angular_z:.2f}, обмежено до: {self.max_angular_z:.2f}'
            )
            angular_z = max(-self.max_angular_z, min(self.max_angular_z, angular_z))

        #Виведення прийнятих (та обмежених) значень через INFO-лог
        self.get_logger().info(f'Отримано команди -> linear.x: {linear_x:.2f}, angular.z: {angular_z:.2f}')
```

Клас ноди включає:

1. Підписку на топік /cmd_vel для задання швидкостей.
2. Встановлення граничних значень швидкостей self.max_linear_x та self.max_angular_z.
3. Перевірку обмежень швидкостей з виведенням в лог WARN у разі перевищення граничних значень.
4. Виведення прийнятих та обмежених значень в лог INFO

2) Тестуємо ноду.

2.1) Створення package:

```
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk/Lab_2$ ros2 pkg create --build-type amen
t_python motion_control --dependencies rclpy geometry_msgs
going to create a new package
package name: motion_control
destination directory: /home/alexmazila/Documents/PDS_KSMm-26_Pavliuk/Lab_2
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['alexmazila <alexpavluk10@gmail.com>']
licenses: ['TODO: License declaration']
build type: ament_python
dependencies: ['rclpy', 'geometry_msgs']
creating folder ./motion_control
creating ./motion_control/package.xml
creating source folder
creating folder ./motion_control/motion_control
creating ./motion_control/setup.py
creating ./motion_control/setup.cfg
creating folder ./motion_control/resource
creating ./motion_control/resource/motion_control
creating ./motion_control/motion_control/__init__.py
creating ./motion_control/motion_control/py.typed
creating folder ./motion_control/test
creating ./motion_control/test/test_copyright.py
creating ./motion_control/test/test_flake8.py
creating ./motion_control/test/test_mypy.py
creating ./motion_control/test/test_pep257.py
creating ./motion_control/test/test_xmllint.py

[WARNING]: Unknown license 'TODO: License declaration'.  This has been set in the package.
xml, but no LICENSE file has been created.
It is recommended to use one of the ament license identifiers:
Apache-2.0
BSL-1.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
MIT
MIT-0
```

2.2) Запуск ноди:

```
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk$ source install/setup.bash
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk$ ros2 run motion_control motion_node
[INFO] [1791715255.227427223] [motion_control_node]: Motion Control Node успішно запущено
та підписано на /cmd_vel
```

2.3) Задаємо значення швидкостей вище граничних за допомогою ros2 topic pub:

```
alexmazila@Ubuntu:~$ ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist "{linear: {x:
1.2, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 2.0}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=1.2, y=0.0, z=0.
0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=2.0))

alexmazila@Ubuntu:~$
```

2.4) Отримуємо WARN повідомлення:

```
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk$ ros2 run motion_control motion_node
[INFO] [1791715255.227427223] [motion_control_node]: Motion Control Node успішно запущено
та підписано на /cmd_vel
[WARN] [1791715283.060751228] [motion_control_node]: Лінійна швидкість перевищує ліміт! От
римано: 1.20, обмежено до: 0.50
[WARN] [1791715283.061201977] [motion_control_node]: Кутова швидкість перевищує ліміт! Отр
имано: 2.00, обмежено до: 1.00
```

2.5) Вводимо допустимі значення швидкостей:

```
alexmazila@Ubuntu:~$ ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist "{linear: {x:
0.2, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.0}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.2, y=0.0, z=0.
0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

alexmazila@Ubuntu:~$
```

2.6) Отримуємо відповідні записи в лог

```
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk$ ros2 run motion_control motion_node
[INFO] [1791715255.227427223] [motion_control_node]: Motion Control Node успішно запущено
та підписано на /cmd_vel
[WARN] [1791715283.060751228] [motion_control_node]: Лінійна швидкість перевищує ліміт! От
римано: 1.20, обмежено до: 0.50
[WARN] [1791715283.061201977] [motion_control_node]: Кутова швидкість перевищує ліміт! Отр
имано: 2.00, обмежено до: 1.00
[INFO] [1791715283.061664015] [motion_control_node]: Отримано команди -> linear.x: 0.50, a
ngular.z: 1.00
[INFO] [1791715426.025756660] [motion_control_node]: Отримано команди -> linear.x: 0.20, a
ngular.z: 1.00
```

2.7) Паралельна перевірка топіку через ros2 topic echo

```
alexmazila@Ubuntu:~$ ros2 topic echo
linear:
  x: 1.2
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 2.0
---
linear:
  x: 0.2
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0
---
```

3) Висновки

На цій лабораторній роботі ми розробили ноду керування рухом робота через /cmd_vel.

На основі проведеного тестування у симуляторі було виявлено ключові різниці між ручним та програмним керуванням:

1. Ручне керування (наприклад, через стандартну ноду teleop_twist_keyboard схоже на ноду як в черепашці, але з динамічною швидкістю) забезпечує безпосередній інтерактивний контроль оператором у реальному часі, проте воно є вразливим до людського фактора та непередбачуваних стрибків команд.
2. Програмне керування дозволяє автоматизувати процес переміщення, гарантує точне дотримання заданих алгоритмів і надійність завдяки вбудованим механізмам фільтрації та обмеження швидкості.

Реалізація обмежень швидкостей доводить важливість введення мір безпеки для захисту роботів в майбутніх проєктах.

Матеріали роботи було структуровано та опубліковано на GitHub https://github.com/alexmazila/PDS_KSMm-26_Pavliuk/tree/main